3. *pygame.mouse.get_pressed():* It returns a Boolean value for every mouse button (1 if a button is clicked, otherwise 0), in a tuple of three elements.
4. There is a problem with the inbuilt rotate function: the rotated surface doesn't have the same dimensions as the original image; so we can not blit the returned surface into the original image because that will lead to flickering, as well as wrong imposition of the surface. So we cut out the centre of the returned surface, from the co-ordinates where we want to draw. In this way, the centre of every image coincides, and we get the proper rotation.
5. We use the *transform* module and its *rotate* function, which takes the image to be rotated, and the angle, as arguments. It returns the rotated surface, which we stored in the variable *image2*. If the angle is positive, it will rotate anticlockwise, and if negative, then clockwise.

The *transform* module is of great use. It has functions like *scaling()*, *flip()* etc, which are required in almost every game. Explore more about them on the Web.

Sprite sheets

This is one of my favourite topics, and I bet will become one of yours too—after reading about it. First of all, animation is nothing but a continuous, fast display of sprite images from sprite sheets. You can also define animation as a collection of images in a serial order. For an example, look at Figure 4, consisting of 7 images. When they are used in a continuous manner with a high frame-rate, it appears to be an explosion. That's how animation is created. You can create the whole game using a single sprite sheet; it depends on how you use that sprite sheet. You must have a good knowledge of surfaces to work with sprites—that comes with experience. Traversing the sprites from the sprite sheet is what we learn next; there is a special sprites module to do this, but we will start from scratch.

```
#sprite.py
import pygame
from pygame.locals import *
from sys import exit

counter=0

def Update():
 global counter
 counter=(counter+1)%7

def sprite(w, h):
    a=[]
    clock=pygame.time.Clock()
    screen=pygame.display.set_mode((200,200),0,24)
    image = pygame.image.load("Image4.png").convert_alpha()
    width, height=image.get_size()
    for i in xrange(int(width/w)):
        a.append(image.subsurface((i*w,0, w, h)))
    while True:
     for i in pygame.event.get():
      if i.type==QUIT:
       exit()
     screen.fill((0,0,0))
     screen.blit(a[counter],(100,100))
     Update()
     pygame.display.update()
     clock.tick(5)
sprite(20,20)
```

If you are good with the GIMP, you will find traversing easy, because it is just a game of giving dimensions. In the above script, the sprite sheet used has a width of 140 and height of 20 pixels. We have 7 properly and equally spaced images, so let's have a width of 20 pixels for each image. Let's work with a rectangular (square, in this case) surface of 20*20 for each image, and then we will store them in a list. *Image4.png* must be in the same directory as the script—otherwise, use *os.path.join* to construct the path. We have created a function called *sprite()* to cut the image into sub-surfaces of 20*20, and then store them in the list 'a' in a continuous manner. Then we will display them by incrementing the *counter* value, which is done in the *Update()* function. That's it—the animation is done, and you will see the 'blast' on the output when you run the script.

Hungry-snakes

I believe that after reading both parts of this series, you can build a small game. For game development, control over loops is a must. I turned the famous snake game (of Nokia fame) into a 2D practice game, and released it as an open source project on Google, so that others could use it to learn, and modify it in their own way. I named it 'Hungry-snakes', and released it under GPL v3. Get it from *http://code.google.com/p/hungry-snakes/downloads/list*. As of writing this, 450+ downloads have already happened. Feel free to ping me for suggestions and queries. END

By: Ankur Aggarwal

The author loves to explore open source technologies, and enjoys programming. He has learned C, C++, core Java, and Python. He loves rock/metal music, and has been trying his hand at the guitar. He has also started a blog recently (*www.flossstuff.wordpress.com*) and can be contacted via *coolankur2006@gmail.com*, or on Twitter as *@ankurtwi*.